

Source3: A Rule-Based Pipeline for Equation Annotation in Scientific Papers

1 Overview

1.1 Problem Analysis

The task is to build a generally applicable system for quantum physics papers from arXiv. For every paper in the assigned list, the system must identify the important numbered equations, extract their meaning, explain the symbols and abbreviations used in them, and describe relations between equation pairs in a graph-ready JSON format. Each output entry also needs an audit trail so that the extracted information can be traced back to the source document.

Several approaches are possible. A purely rule-based system is transparent and easy to audit, but brittle when papers use different notation or wording. A retrieval-based system using BM25 or dense scientific embeddings can find relevant explanatory sentences more flexibly, but still needs filters to avoid weak matches. Generative language models could summarise equations, but they are less suitable here because the dataset requires traceable evidence from the paper. Source3 therefore combines deterministic document parsing, lexical and semantic retrieval, and hand-written extraction patterns to keep the prototype both general enough for arbitrary papers and auditable enough for the required dataset.

1.2 Approach Overview

Source3 is a deterministic, non-generative pipeline built around a shared embedding space. After downloading and parsing the arXiv HTML, the system extracts numbered equations, nearby sentences, cross-references, sections, and MathML symbol structure. Equations and text chunks are then embedded with MathBERT [1] ([witiko/mathberta](#)), so that mathematical expressions and their surrounding explanations can be compared by cosine similarity.

The same embedding representation is used for the three main extraction tasks.

- **Equation meaning:** the equation and its context are matched against candidate explanatory sentences from the paper.
- **Symbol meaning:** each symbol is used as a query to retrieve likely definition sentences, which are then checked with simple linguistic patterns such as “where x is” or “ x denotes”.
- **Relations:** pairs of equations are compared through their embedded contexts, shared symbols, section distance, and explicit cue phrases such as references to earlier equations or derivation statements.

This design keeps the system general enough for different quantum physics papers while remaining auditable. Embeddings provide flexible retrieval, and deterministic rules select only source-backed spans and record the evidence used in the final JSON.

1.3 Pipeline Diagram

The diagram below shows the data flow through the ten stages.



2 System Architecture

This section describes the internal architecture from document parsing to embedding construction (Stages 2–5). These stages are important because the later extraction tasks cannot work directly on raw HTML. ArXiv HTML mixes readable text, MathML equation trees, visible equation numbers, section anchors, navigation links, and bibliography entries into a single DOM tree. Stages 2–5 transform that noisy structure into increasingly task-ready representations: a clean structured document, typed retrieval chunks, selected equation records, and finally a shared embedding space. The examples below use paper 2401.00059.

2.1 Stage 2: Document Parsing

Responsible file: `source3/stage02_build_documents.py`

Output: `data/pipeline/documents/<paper_id>.json`.

How it works. The parser uses BeautifulSoup to traverse the DOM and performs five sub-tasks:

1. **Section tree.** Every `div` whose CSS class belongs to the set `{ltx_section, ltx_subsection, ltx_subsubsection, ltx_paragraph, ltx_appendix}` is turned into a `SectionRecord`. A stack tracks nesting so each section knows its parent. A synthetic *preamble* section captures text that appears before the first heading. Sections whose class is `ltx_bibliography` are excluded.
2. **Paragraphs and sentences.** Each `<p class="ltx_p">` that is not inside a figure, table, navigation block, or bibliography is cleaned: inline `<math>` tags are replaced by their `alttext` attribute (the $\text{\LaTeX}$ fallback), then the paragraph is split into sentences. Sentence splitting uses spaCy's `sentencizer` when it is installed and a hand-written regex as a fallback. The regex handles common physics abbreviations such as *Eq.*, *Ref.*, and *cf.* to avoid incorrect sentence breaks.
3. **Equation extraction.** All `<table class="ltx_equation">` elements are collected in DOM order. For each table the parser reads the $\text{\LaTeX}$ source from the `<annotation encoding="application/x-tex">` child, the HTML anchor from the table's `id` attribute, and the visible equation number from the `ltx_eqn_eqno` element. The full MathML subtree is also serialized and stored for symbol extraction in Stage 7.
4. **Neighbourhood linking.** For each equation table the parser uses a binary search on a position-sorted paragraph list to find the up to five prose sentences immediately before and after the equation in reading order. These sentence IDs are stored in the equation record and later used to build context windows for retrieval.
5. **Cross-reference extraction.** A regular expression scans every sentence for phrases of the form *Eq. (3)* or *Equations (5)–(8)*. Each match is stored as a `CrossReferenceRecord` carrying the source sentence, character offsets, and the list of target equation labels. Range patterns such as *(5)–(8)* are expanded into the full list `[5, 6, 7, 8]`.

All five sub-results are merged into a single document JSON. Each entity (section, paragraph, sentence, equation, cross-reference) gets a stable hierarchical identifier such as `sec:section:2:p3:s1` so that later stages can cite exact provenance.

```
{
  "paper_id": "2401.00059",
  "sentences": {
    "sec:section:2:p2:s6": {
      "text": "We write down the motion equation in the form",
      "paragraph_id": "sec:section:2:p2",
      "cross_reference_ids": []
    }
  },
}
```

```

"raw_equations": [{
  "raw_equation_id": "raw:0001",
  "anchor_id": "S2.E1",
  "visible_labels": ["1"],
  "latex": "\\ddot{u}_{n}=-\\beta(u_{n}-u_{n-1})-...",
  "before_sentence_ids": ["sec:section:2:p2:s6"],
  "after_sentence_ids": ["sec:section:2:p3:s1"]
}]
}

```

2.2 Stage 3: Chunk Construction

Responsible file: source3/stage03_build_chunks.py

Output: data/pipeline/chunks/<paper_id>.json.

How it works. The chunk builder reads the document from Stage 2 and produces six chunk types.

- **sentence** — one chunk per sentence. Used for precise, word-level evidence in symbol definition lookup.
- **paragraph** — one chunk per paragraph. Used when a meaning requires broader local context.
- **section_context** — all paragraphs in a section concatenated. Used to retrieve section-level explanations and background.
- **raw_equation** — the $\text{\LaTeX}$ string of the equation alone. Used to search for equations that share notation.
- **raw_equation_neighborhood** — the up-to-five sentences before and after the equation sandwiched around the $\text{\LaTeX}$ string, formatted as `[EQUATION: \dots]` in the middle. This is the primary retrieval unit for equation meaning because it captures the immediate textual context in which the equation appears.
- **cross_reference** — the sentence that explicitly mentions an equation by number. Used in relation detection to find cue phrases such as “substituting Eq. (3) into Eq. (5)”.

Every chunk records a stable `chunk_id`, the source sentence or paragraph IDs it was built from, the section title, and the document order of its anchor paragraph. This metadata allows later stages to score evidence not only by semantic similarity but also by proximity to the target equation.

```

{
  "chunk_id": "2401.00059:raw_equation_neighborhood:raw:0001",
  "chunk_type": "raw_equation_neighborhood",
  "raw_equation_anchor": "S2.E1",
  "document_order": 412,
  "section_title": "Model",
  "source_ids": {
    "raw_equation_id": "raw:0001",
    "before_sentence_ids": ["sec:section:2:p2:s6"],
    "after_sentence_ids": ["sec:section:2:p3:s1"]
  },
  "text": "We write down the motion equation in the form
[EQUATION: \\ddot{u}_{n}=...] where we omitted ..."
}

```

2.3 Stage 4: Equation Extraction

Responsible file: source3/stage04_extract_equations.py

Output: data/pipeline/equations/<paper_id>.json.

Why this stage is necessary. Stage 2 stores every display equation found in the HTML, including unnumbered derivation steps, auxiliary relations, and table-formatted equations that are not independently meaningful. The task specification asks only for the important *numbered* equations up to a configured cap per paper. Stage 4 therefore selects the canonical dataset equations and builds the clean records that Stages 6–9 will annotate.

How it works. The script iterates over papers in the order given by the paper list. A global counter stops processing once the total equation budget across all papers is reached; this ensures the dataset stays within the assignment’s target size. Within each paper, it takes at most MAX_EQUATIONS_PER_PAPER raw equations from the front of the document-order list.

For each selected raw equation it builds an **equation record** that carries:

- `equation_id` — the sequential number used as the key in the final dataset JSON (“1”, “2”, ...);
- `latex` — the normalized $\text{\LaTeX}$ string (whitespace collapsed, `\displaystyle` stripped);
- `raw_equation_id` — a link back to the Stage 2 raw equation record;
- `anchor_id` — the HTML element `id` that uniquely identifies this equation in the source document;
- `before_sentences` / `after_sentences` — the context window sentences copied from Stage 2, with their full text resolved so later stages do not need to re-load the document.

```
{
  "paper_id": "2401.00059",
  "equations": [{
    "equation_id": "1",
    "latex": "\\ddot{u}_{n}=-\\beta(u_{n}-u_{n-1})-...",
    "raw_equation_id": "raw:0001",
    "anchor_id": "S2.E1",
    "section_id": "sec:section:2",
    "document_order": 412,
    "before_sentences": [{
      "sentence_id": "sec:section:2:p2:s6",
      "text": "We write down the motion equation in the form"
    }],
    "after_sentences": [{
      "sentence_id": "sec:section:2:p3:s1",
      "text": "where we omitted higher-order terms."
    }]
  }]
}
```

2.4 Stage 5: Embedding Construction

Responsible file: source3/stage05_build_embeddings.py

Output: data/pipeline/embeddings/<paper_id>.json

and data/pipeline/embeddings/<paper_id>.npz.

Why this stage is necessary. Equation meaning, symbol definitions, and relations are all retrieval problems that cannot be solved by exact keyword matching. Authors may describe the same equation with different vocabulary, and a symbol’s definition may appear several sentences away from its first use. By projecting both equations and text into the same 768-dimensional vector space using

MathBERT (witiko/mathberta), the pipeline can compare them by cosine similarity and retrieve the most relevant explanation even when no word overlaps.

Model and pooling. MathBERT is a BERT-based model pre-trained on a corpus that includes mathematical notation. The script loads the model and tokenizer from HuggingFace, moves them to GPU if available, and processes inputs in batches of 32 to keep memory bounded. Given a batch of input strings, the model returns a `last_hidden_state` tensor of shape $(B, L, 768)$, where B is the batch size and L is the sequence length. The script pools the hidden states using **attention-mask mean pooling**: it multiplies each token's hidden state by its attention mask value (0 for padding, 1 for real tokens), sums across the sequence dimension, and divides by the number of real tokens. This avoids letting padding tokens influence the sentence vector. Each resulting 768-dimensional vector is then ℓ_2 -normalized so that dot-product search equals cosine similarity.

Two embedding families are built.

Family A – equation-centric vectors. For each equation selected in Stage 4, the script builds several vectors that capture the equation from different angles:

- **equation** — the $\text{\LaTeX}$ string alone, prefixed with the token `equation:`. Captures the mathematical content without any textual bias.
- **summary** — the equation flanked by its surrounding sentences, formatted as `before: ... \n equation: ... \n after: ....`. This is the primary query vector for meaning retrieval because it blends the math with its immediate prose context.
- **before_sentence / after_sentence** — one vector per neighbouring sentence, formatted as `equation: ... \n context: ... \n position: before|after \n distance: N`. Distance is 1 for the immediately adjacent sentence, 2 for the next, and so on. These allow Stage 6 to score individual nearby sentences against the equation.

All Family A input strings are token-budget-managed: if the $\text{\LaTeX}$ alone exceeds the model's 512-token limit, the equation is truncated symmetrically from both ends with a `[...]` marker, and any remaining budget is given to context.

Family B – sentence-chunk vectors. For every **sentence** chunk and every **raw_equation_neighborhood** chunk from Stage 3, the script builds one vector using head truncation (first N tokens). These vectors represent the full paper's text and are used as the candidate pool that later stages search against.

The actual float32 matrix of shape $(N_{\text{rows}}, 768)$ is saved as a compressed `.npz` file. A JSON sidecar records every row's `vector_id`, `vector_kind`, `source_text`, row index, token count, and truncation flags so that any result can be traced back to the exact input text and equation it came from.

```
{
  "paper_id": "2401.00059",
  "model": "witiko/mathberta",
  "pooling": "attention_mask_mean",
  "normalized": true,
  "dimension": 768,
  "rows": [
    {
      "vector_id": "2401.00059:equation:1:equation",
      "equation_id": "1",
      "vector_kind": "equation",
      "source_text": "equation: \\ddot{u}_{n}=-\\beta(u_{n}-u_{n-1})-...",
      "row": 0
    },
    {
      "vector_id": "2401.00059:equation:1:summary",
      "equation_id": "1",
```

```
    "vector_kind": "summary",
    "source_text": "before: We write down ...\\nequation: ...\\nafter: ...",
    "row": 1
  }
]
```

References

- [1] S. Peng, K. Yuan, L. Gao, and Z. Tang, “MathBERT: A Pre-trained Model for Mathematical Formula Understanding,” *arXiv preprint arXiv:2105.00377*, 2021.